Course 3: Unsupervised Learning



Course 3: Unsupervised Learning



Summary

Last session

1. Supervised learning - learning from labeled examples
2. Challenges of supervised learning
3. Evaluation: ground truth and metrics

Today's session

1. Learning from unlabeled examples
   1. Clustering
   2. Dimensionality reduction (PCA, t-SNE)
2. Evaluating without labels
3. A note on data cleaning

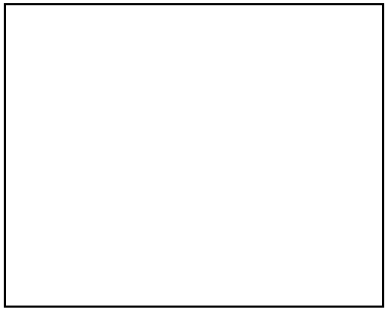
Summary
Last session
Supervised learning - learning from labeled examples
Challenges of supervised learning
Evaluation: ground truth and metrics
Today's session
Learning from unlabeled examples
Clustering
Dimensionality reduction (PCA, t-SNE)
Evaluating without labels
A note on data cleaning

Last session was about *supervised* learning: pairs (input, label), and the goal of predicting the label of inputs never seen. Today the labels disappear.

The question is no longer "what is the right answer for this point?" but "how is this data organised?". Two kinds of methods answer it (clustering and dimensionality reduction), followed by two transversal topics: how to evaluate when there is no ground truth, and how to prepare your features.

Notations

Vector space ($\mathbb{R}^d$)



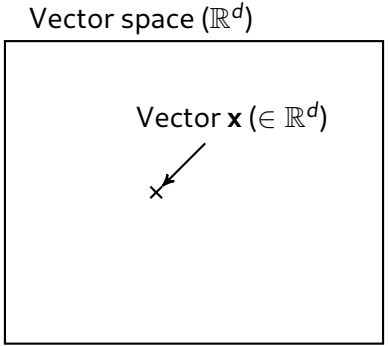
Notations
Vector space ($\mathbb{R}^d$)



Same notations as last session, with one difference: there is no *y* anymore. A dataset is a *set X* of *N* vectors in dimension *d*. Everything we do today uses only the geometry of this cloud: which points are close to which, in which directions the cloud is stretched, whether it lies on a curve.

Keep in mind that "close" depends on a distance, and that this distance depends on the units of your features. We will cover feature normalization in the last part of this course.

Notations

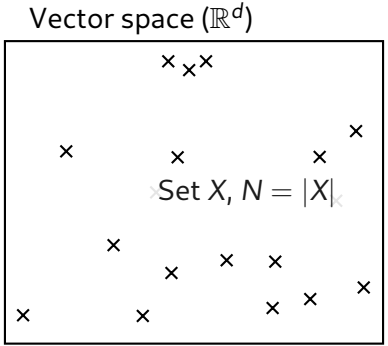


Notations
Vector space ($\mathbb{R}^d$)
Vector $\mathbf{x}$ ($\in \mathbb{R}^d$)

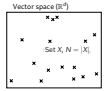
Same notations as last session, with one difference: there is no y anymore. A dataset is a *set* X of N vectors in dimension d. Everything we do today uses only the geometry of this cloud: which points are close to which, in which directions the cloud is stretched, whether it lies on a curve.

Keep in mind that "close" depends on a distance, and that this distance depends on the units of your features. We will cover feature normalization in the last part of this course.

Notations



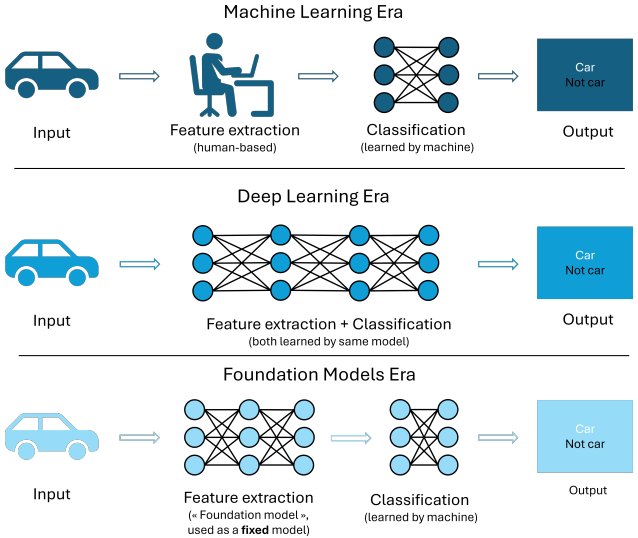
Notations



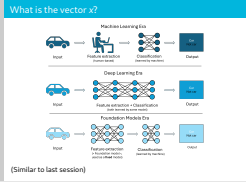
Same notations as last session, with one difference: there is no y anymore.
A dataset is a *set* X of N vectors in dimension d. Everything we do today uses only the geometry of this cloud: which points are close to which, in which directions the cloud is stretched, whether it lies on a curve.

Keep in mind that "close" depends on a distance, and that this distance depends on the units of your features. We will cover feature normalization in the last part of this course.

What is the vector x?



(Similar to last session)



In traditional Machine Learning, data is generally a feature representation of the data (i.e. the edges or the colorimetry levels in an image, or spectrograms in audio).

A first shift appears with Deep Learning, where data is fed as it is to models, allowing to alleviate the complex, time-consuming and hand-crafted feature engineering, and leading to impressive performance in image processing, the drawback being a need for large amounts of data and computing power.

Finally, a recent shift appear with larger and more sophisticated models ("Foundation Models"), where they can be used as a first pre-processing step.

The resulting representations are called "embeddings", and are vectors in the latent space of the model, i.e. the input vectors (image, text, ...) that have been mapped in a lower dimensional space by a function $f : R^d \rightarrow R^l$. Embeddings are usually richer semantically and easier to manipulate for a downstream task (e.g. classification).

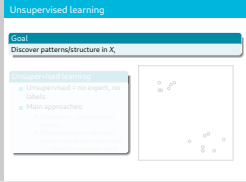
Unsupervised learning

Goal

Discover patterns/structure in X,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering: group similar points,
  - Dimensionality reduction: fewer coordinates per point, → Useful to visualize data!



Two ways of saying "there is structure in this cloud".
Clustering answers *which points go together*: the answer is one integer per point. Dimensionality reduction answers *how to describe each point with fewer numbers*: here, even if the points are 2D, they all lie close to a line (1D), so a single coordinate almost describes them.

The frontier between the two is not strict: replacing a point by the index of its cluster is itself an extreme form of reduction. All these methods reduce the amount of information needed to describe a point, which is why the same two applications will keep coming back: compression and visualization.

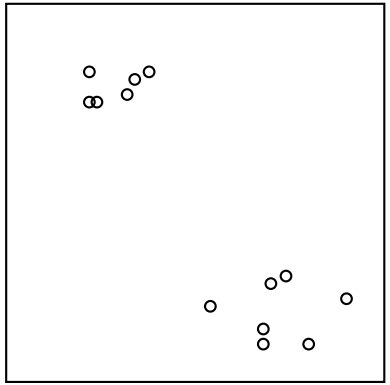
Unsupervised learning

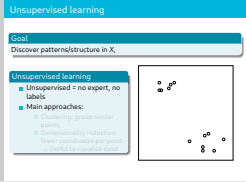
Goal

Discover patterns/structure in X,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering: group similar points,
  - Dimensionality reduction: fewer coordinates per point. → Useful to visualize data!





Two ways of saying “there is structure in this cloud”.
Clustering answers *which points go together*: the answer is one integer per point. Dimensionality reduction answers *how to describe each point with fewer numbers*: here, even if the points are 2D, they all lie close to a line (1D), so a single coordinate almost describes them.

The frontier between the two is not strict: replacing a point by the index of its cluster is itself an extreme form of reduction. All these methods reduce the amount of information needed to describe a point, which is why the same two applications will keep coming back: compression and visualization.

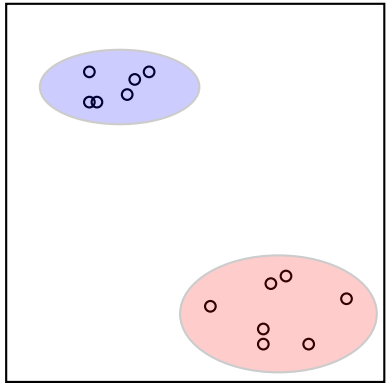
Unsupervised learning

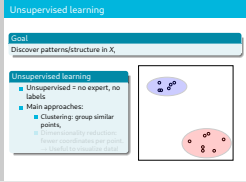
Goal

Discover patterns/structure in X,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering: group similar points,
  - Dimensionality reduction: fewer coordinates per point. → Useful to visualize data!





Two ways of saying "there is structure in this cloud".
Clustering answers *which points go together*: the answer is one integer per point. Dimensionality reduction answers *how to describe each point with fewer numbers*: here, even if the points are 2D, they all lie close to a line (1D), so a single coordinate almost describes them.

The frontier between the two is not strict: replacing a point by the index of its cluster is itself an extreme form of reduction. All these methods reduce the amount of information needed to describe a point, which is why the same two applications will keep coming back: compression and visualization.

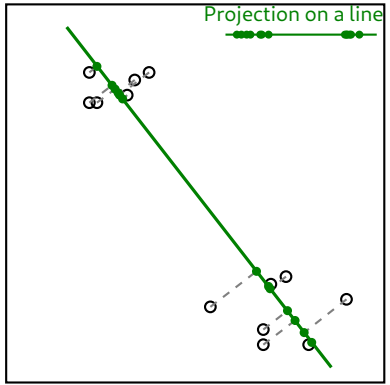
Unsupervised learning

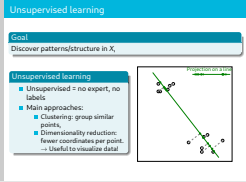
Goal

Discover patterns/structure in X,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering: group similar points,
  - Dimensionality reduction: fewer coordinates per point. $\rightarrow$ Useful to visualize data!





Two ways of saying “there is structure in this cloud”.
Clustering answers *which points go together*: the answer is one integer per point. Dimensionality reduction answers *how to describe each point with fewer numbers*: here, even if the points are 2D, they all lie close to a line (1D), so a single coordinate almost describes them.

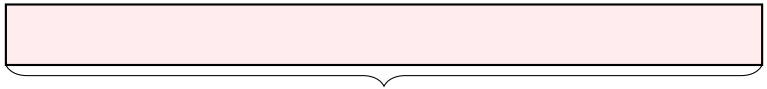
The frontier between the two is not strict: replacing a point by the index of its cluster is itself an extreme form of reduction. All these methods reduce the amount of information needed to describe a point, which is why the same two applications will keep coming back: compression and visualization.

Training and Test sets?

Full Dataset in train/test sets (as for supervised):

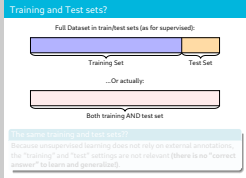


...Or actually:



The same training and test sets??

Because unsupervised learning does not rely on external annotations, the "training" and "test" settings are not relevant **(there is no "correct answer" to learn and generalize!)**.



One of the main practical differences with last session, and a classical source of confusion: since no label is used, nothing prevents you from fitting the model on all the data you have.

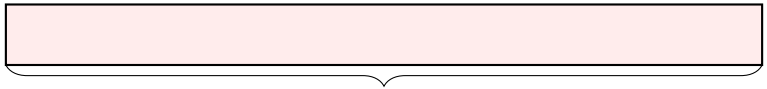
Two nuances. `fit` and `transform` remain separate in scikit-learn, so a fitted model *can* be applied to new points (projecting a new image with an existing PCA, for instance). This matters when unsupervised learning is only a preprocessing step before a supervised model: fit it on the training set only, otherwise information leaks from the test set. And some methods, `TSNE` in particular, have no `transform` at all; they only embed the points they were given.

Training and Test sets?

Full Dataset in train/test sets (as for supervised):

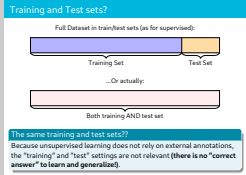


...Or actually:



The same training and test sets??

Because unsupervised learning does not rely on external annotations, the "training" and "test" settings are not relevant **(there is no "correct answer" to learn and generalize!)**.



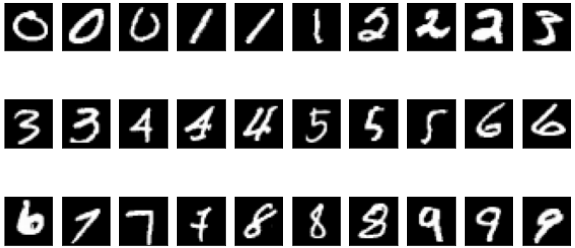
One of the main practical differences with last session, and a classical source of confusion: since no label is used, nothing prevents you from fitting the model on all the data you have.

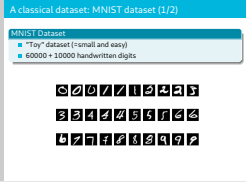
Two nuances. `fit` and `transform` remain separate in scikit-learn, so a fitted model *can* be applied to new points (projecting a new image with an existing PCA, for instance). This matters when unsupervised learning is only a preprocessing step before a supervised model: fit it on the training set only, otherwise information leaks from the test set. And some methods, `TSNE` in particular, have no `transform` at all; they only embed the points they were given.

A classical dataset: MNIST dataset (1/2)

MNIST Dataset

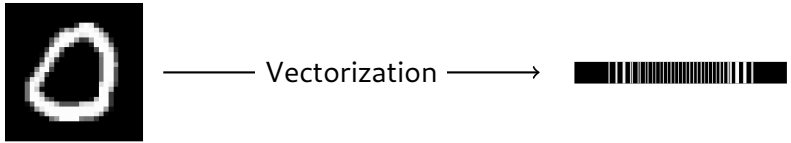
- "Toy" dataset (=small and easy)
- 60000 + 10000 handwritten digits





A dataset that looks a little more like real data. MNIST is a small dataset of handwritten digits: 60000 examples for training and 10000 for test. It used to be an important benchmark, but it is considered far too easy today, hence the name “toy” dataset.

A classical dataset: MNIST dataset (2/2)



Hence, all images are interpreted as 1D vectors!

A classical dataset: MNIST dataset (2/2)
Vectorization
Hence, all images are interpreted as 1D vectors!

This conditions everything that follows: a 28×28 image becomes a vector of $d = 784$ numbers, and the algorithm knows nothing about the 2D layout of the pixels. Two images are "close" if they have similar grey levels at the *same* pixel positions.

Ask yourself what happens if all the digits are shifted two pixels to the right. They look identical to you, but their vectors are now far apart. This is a strong limitation of the methods seen today, and one of the motivations for the representations of the next sessions.

Plan

1 Clustering

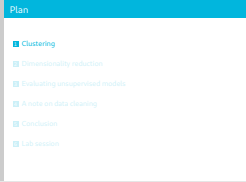
2 Dimensionality reduction

3 Evaluating unsupervised models

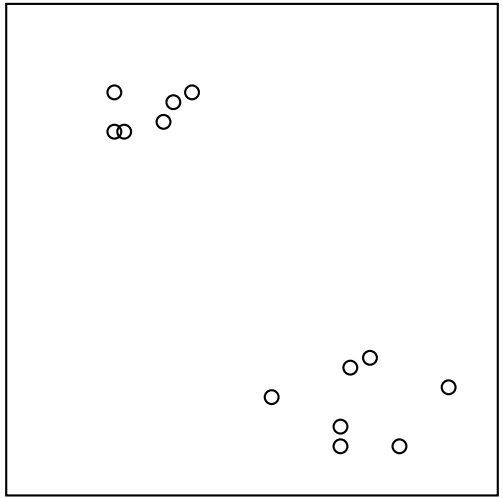
4 A note on data cleaning

5 Conclusion

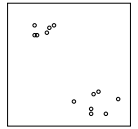
6 Lab session



Clustering: the intuition

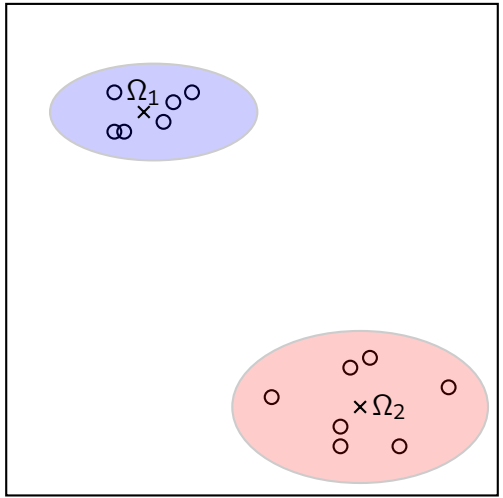


Clustering: the intuition

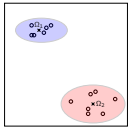


Here is a visual example. If we have the following set of points, then the two centroids Ω_1 and Ω_2 would be reasonable candidates for a clustering with two clusters.

Clustering: the intuition



Clustering: the intuition



Here is a visual example. If we have the following set of points, then the two centroids Ω_1 and Ω_2 would be reasonable candidates for a clustering with two clusters.

Clustering with centroids

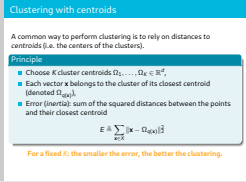
A common way to perform clustering is to rely on distances to *centroids* (i.e. the centers of the clusters).

Principle

- Choose K cluster centroids $\Omega_1, \ldots, \Omega_K \in \mathbb{R}^d$,
- Each vector $\mathbf{x}$ belongs to the cluster of its closest centroid (denoted $\Omega_{q(\mathbf{x})}$),
- Error (*inertia*): sum of the squared distances between the points and their closest centroid

$$E \triangleq \sum_{\mathbf{x} \in X} \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2^2$$

For a fixed K: the smaller the error, the better the clustering.



A definition of clustering using centroids. Other definitions exist, based on regions, on density, or on probabilities.
The point to remember is the error: the sum of the *squared* distances between points and their closest centroid. scikit-learn calls it *inertia* (`model.inertia_`). Note that it is expressed in the units of your features, squared: comparing it between two datasets is meaningless! Only compare it between two clusterings of the same dataset.

In more detail (for the curious): $q : \mathbb{R}^d \to [1..K]$ is the function that associates a vector $\mathbf{x}$ with the index of (one of) its closest centroids, i.e. $\forall j \in [1..K], \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2 \leq \|\mathbf{x} - \Omega_j\|_2$. Cluster k is then the set $\{\mathbf{x} \in X, q(\mathbf{x}) = k\}$, and the K clusters form a partition of X: $X = \bigcup_k \{\mathbf{x} \in X, q(\mathbf{x}) = k\}$.

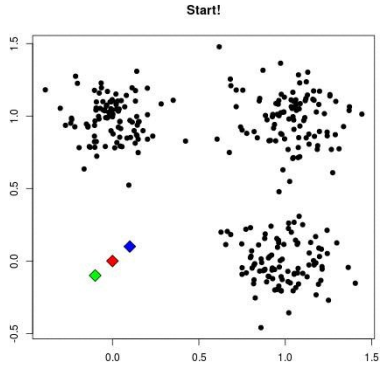
A standard algorithm: K-Means

K-means algorithm

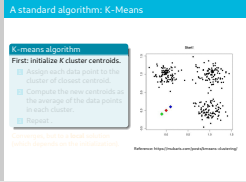
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

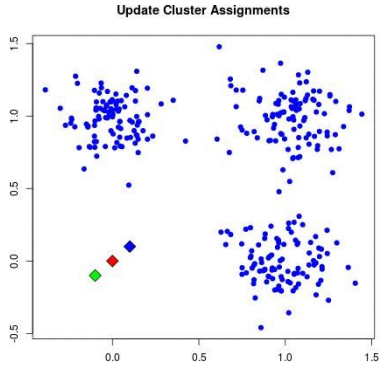
A standard algorithm: K-Means

K-means algorithm

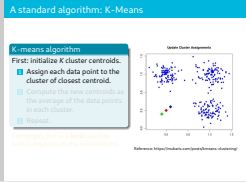
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

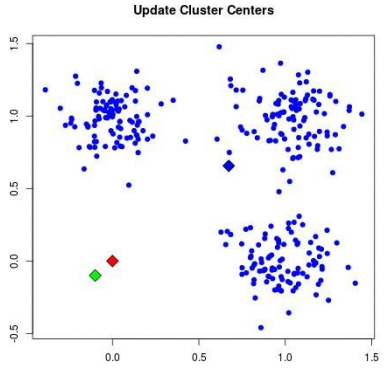
A standard algorithm: K-Means

K-means algorithm

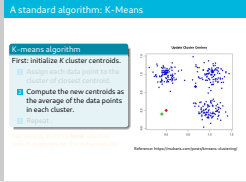
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

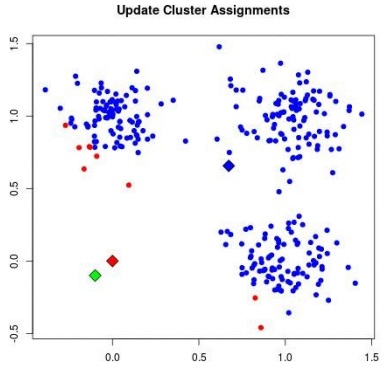
A standard algorithm: K-Means

K-means algorithm

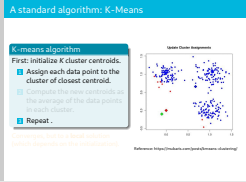
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

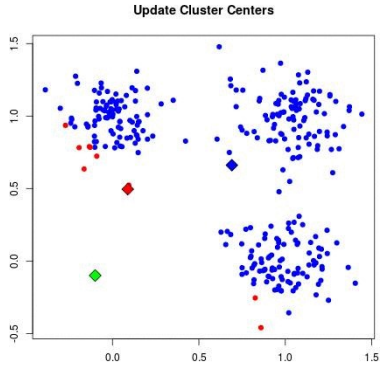
A standard algorithm: K-Means

K-means algorithm

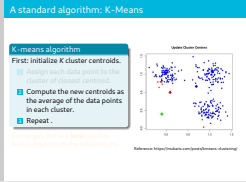
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

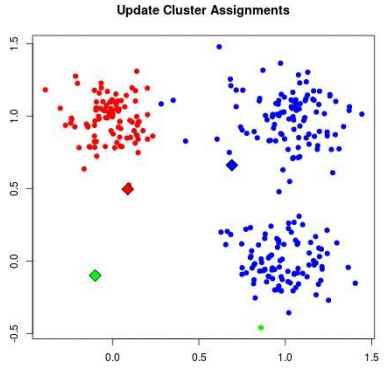
A standard algorithm: K-Means

K-means algorithm

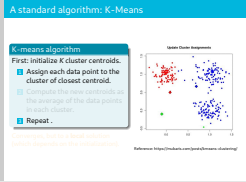
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

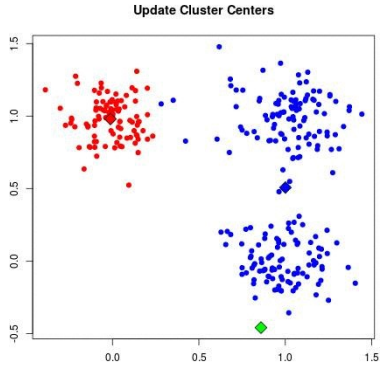
A standard algorithm: K-Means

K-means algorithm

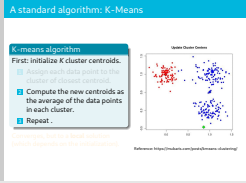
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

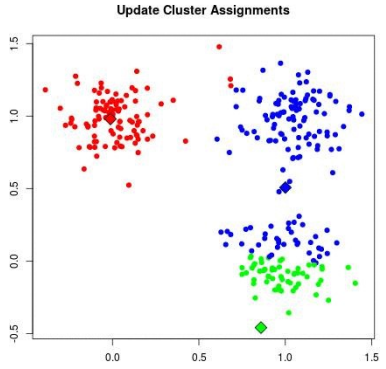
A standard algorithm: K-Means

K-means algorithm

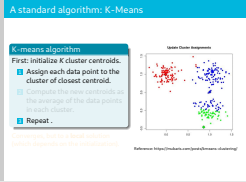
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

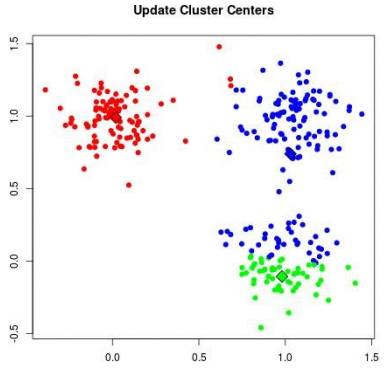
A standard algorithm: K-Means

K-means algorithm

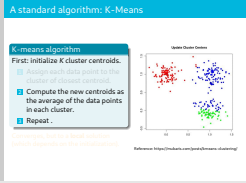
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

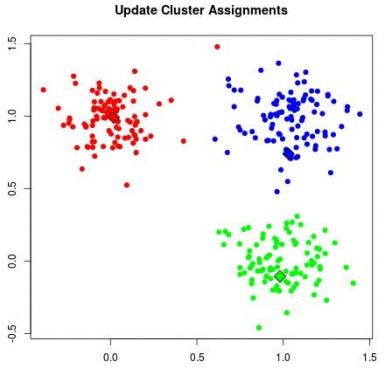
A standard algorithm: K-Means

K-means algorithm

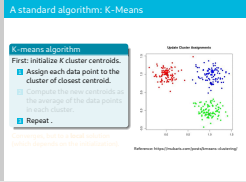
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

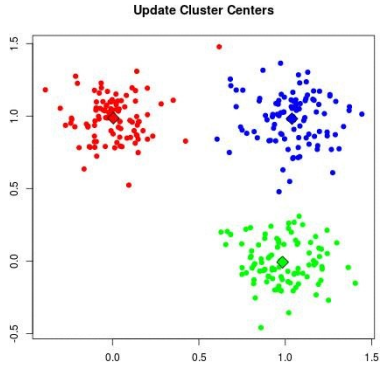
A standard algorithm: K-Means

K-means algorithm

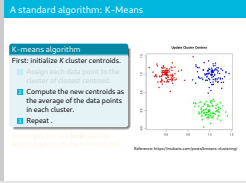
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

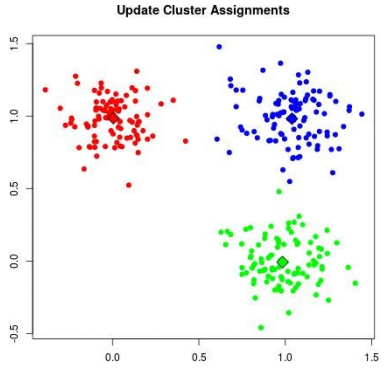
A standard algorithm: K-Means

K-means algorithm

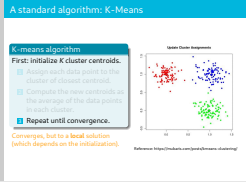
First: initialize K cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat until convergence.

Converges, but to a **local** solution (which depends on the initialization).



Reference: https://mubaris.com/posts/kmeans-clustering/



Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Why not look for the best clustering directly? Because finding the clustering with the smallest possible error is an NP-hard problem: it is not tractable as soon as N grows. K-means never looks for the optimum, only for a decent local solution.

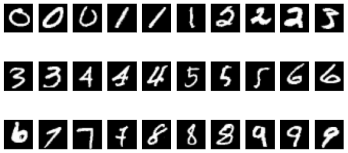
Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

In practice: K-Means on MNIST

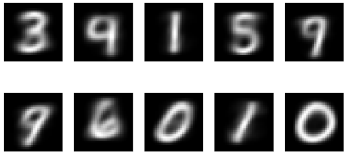
Clustering MNIST

Using K-means algorithm with $K = 10$.
$\rightarrow$ Clusters capture, to some extent, the different digits.

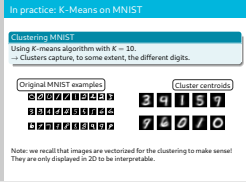
Original MNIST examples



Cluster centroids



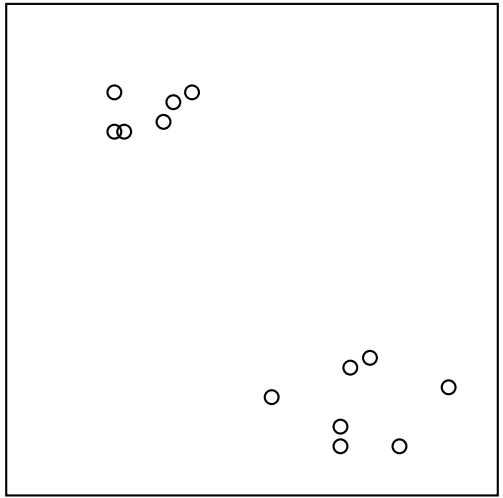
Note: we recall that images are vectorized for the clustering to make sense! They are only displayed in 2D to be interpretable.



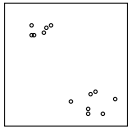
A clustering test on MNIST with the K-means algorithm of the previous slide, run with $K = 10$.

On the left, original examples; on the right, the 10 centroids obtained. Some clusters clearly capture one digit (6, 1, 2, 0), but others split a single digit into several clusters (8, 4, 3). With $K = 10$ there is no guarantee that the 10 clusters match the 10 classes; nothing in the algorithm knows that digits exist.

Impact of the choice of K

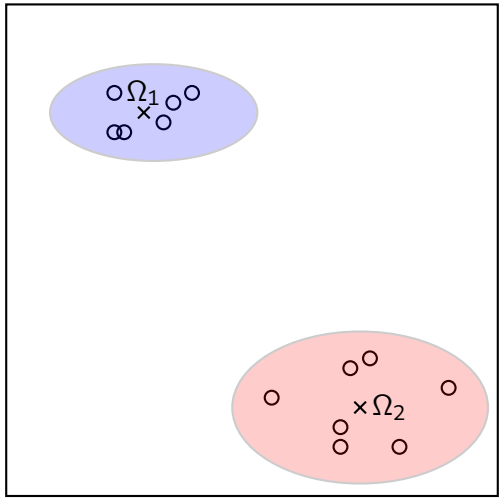


Impact of the choice of K



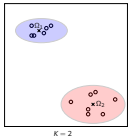
Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with $K = 1$ everything collapses into a single group, with $K = N$ each point is its own cluster and nothing has been learned. Neither is useful, so K has to be chosen.

Impact of the choice of K



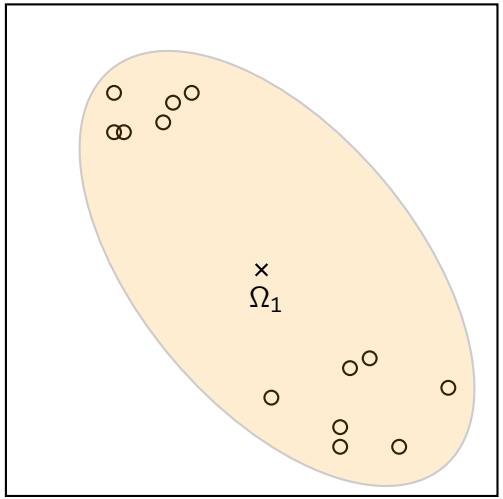
$K = 2$

Impact of the choice of K



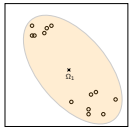
Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with $K = 1$ everything collapses into a single group, with $K = N$ each point is its own cluster and nothing has been learned. Neither is useful, so K has to be chosen.

Impact of the choice of K



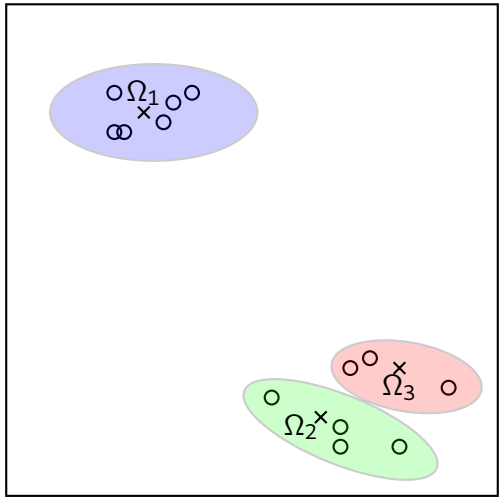
$K = 1$

Impact of the choice of K



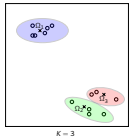
Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with $K = 1$ everything collapses into a single group, with $K = N$ each point is its own cluster and nothing has been learned. Neither is useful, so K has to be chosen.

Impact of the choice of K



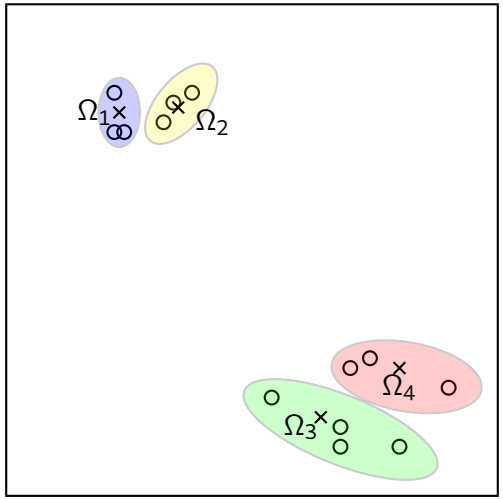
$K = 3$

Impact of the choice of K



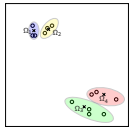
Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with $K = 1$ everything collapses into a single group, with $K = N$ each point is its own cluster and nothing has been learned. Neither is useful, so K has to be chosen.

Impact of the choice of K



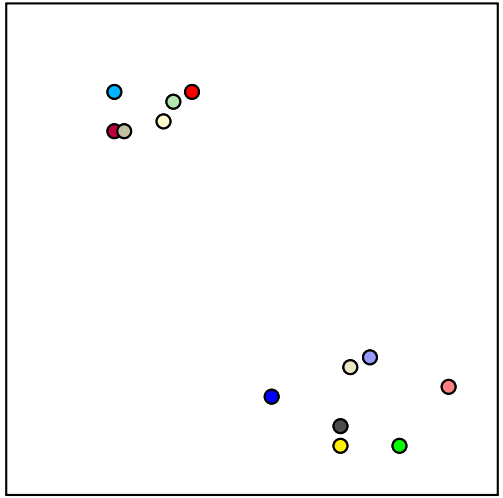
$K = 4$

Impact of the choice of K



Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with $K = 1$ everything collapses into a single group, with $K = N$ each point is its own cluster and nothing has been learned. Neither is useful, so K has to be chosen.

Impact of the choice of K



$K = N$
(each data point is its own centroid)

Impact of the choice of K
$K = N$
(each data point is its own centroid)

Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with $K = 1$ everything collapses into a single group, with $K = N$ each point is its own cluster and nothing has been learned. Neither is useful, so K has to be chosen.

The error alone cannot choose K

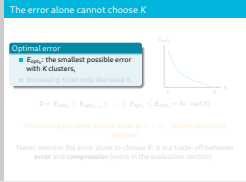
Optimal error

- E_{opt_K}: the smallest possible error with K clusters,
- Increasing K can only decrease it.

$$0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \cdots \leq E_{opt_2} \leq E_{opt_1} = N \cdot var(X)$$

Minimizing the error always leads to $K = N$…where nothing is learned!

Never monitor the error alone to choose K: it is a trade-off between **error** and **compression** (more in the evaluation section).

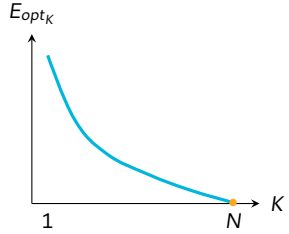


Why you must not simply minimise the loss to choose K.
The extremes are easy to compute. With one centroid per point ($K = N$), the error is 0. With a single cluster ($K = 1$), the best centroid is the average of all points, and the error is N times the variance of X (E is a *sum* of squared distances). In between, adding a cluster never hurts: with $K + 1$ centroids, keep the K best ones and place the extra one on a data point; the error can only go down.
Hence the "best" K according to the error is always $K = N$, which is useless. The compression factor $\kappa = 1 - \frac{K}{N}$ makes the trade-off explicit: large K, small error but no compression ($\kappa \to 0$); small K, the opposite. The heuristics to pick a point on this trade-off (elbow, silhouette) come in the evaluation section.
E_{opt_K} is theoretical: finding the optimal clustering is NP-hard, and K-means only returns a local solution, so the curve you measure may not be perfectly decreasing.

The error alone cannot choose K

Optimal error

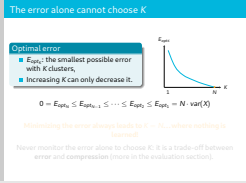
- E_{opt_K}: the smallest possible error with K clusters,
- Increasing K can only decrease it.



$$0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \cdots \leq E_{opt_2} \leq E_{opt_1} = N \cdot var(X)$$

Minimizing the error always leads to $K = N$…where nothing is learned!

Never monitor the error alone to choose K: it is a trade-off between **error** and **compression** (more in the evaluation section).

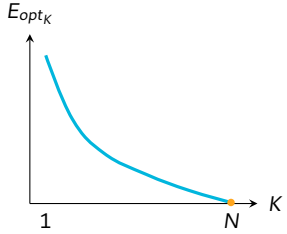


Why you must not simply minimise the loss to choose K.
The extremes are easy to compute. With one centroid per point ($K = N$), the error is 0. With a single cluster ($K = 1$), the best centroid is the average of all points, and the error is N times the variance of X (E is a *sum* of squared distances). In between, adding a cluster never hurts: with $K + 1$ centroids, keep the K best ones and place the extra one on a data point; the error can only go down.
Hence the "best" K according to the error is always $K = N$, which is useless. The compression factor $\kappa = 1 - \frac{K}{N}$ makes the trade-off explicit: large K, small error but no compression ($\kappa \to 0$); small K, the opposite. The heuristics to pick a point on this trade-off (elbow, silhouette) come in the evaluation section.
E_{opt_K} is theoretical: finding the optimal clustering is NP-hard, and K-means only returns a local solution, so the curve you measure may not be perfectly decreasing.

The error alone cannot choose K

Optimal error

- E_{opt_K}: the smallest possible error with K clusters,
- Increasing K can only decrease it.



$$0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \cdots \leq E_{opt_2} \leq E_{opt_1} = N \cdot var(X)$$

Minimizing the error always leads to $K = N$…where nothing is learned!

Never monitor the error alone to choose K: it is a trade-off between **error** and **compression** (more in the evaluation section).

Why you must not simply minimise the loss to choose K.
The extremes are easy to compute. With one centroid per point ($K = N$), the error is 0. With a single cluster ($K = 1$), the best centroid is the average of all points, and the error is N times the variance of X (E is a *sum* of squared distances). In between, adding a cluster never hurts: with $K + 1$ centroids, keep the K best ones and place the extra one on a data point; the error can only go down.
Hence the "best" K according to the error is always $K = N$, which is useless. The compression factor $\kappa = 1 - \frac{K}{N}$ makes the trade-off explicit: large K, small error but no compression ($\kappa \to 0$); small K, the opposite. The heuristics to pick a point on this trade-off (elbow, silhouette) come in the evaluation section.
E_{opt_K} is theoretical: finding the optimal clustering is NP-hard, and K-means only returns a local solution, so the curve you measure may not be perfectly decreasing.

Plan

1 Clustering

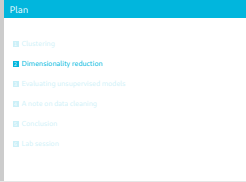
2 Dimensionality reduction

3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session



Why reduce the dimension?

High dimensional data is hard to handle (to visualize, to process, curse of dimensionality):

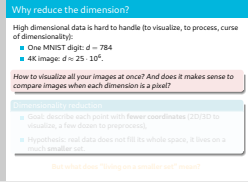
- One MNIST digit: $d = 784$
- 4K image: $d \approx 25 \cdot 10^6$.

How to visualize all your images at once? And does it makes sense to compare images when each dimension is a pixel?

Dimensionality reduction

- Goal: describe each point with **fewer coordinates** (2D/3D to visualize, a few dozen to preprocess),
- Hypothesis: real data does not fill its whole space, it lives on a much **smaller** set.

But what does "living on a smaller set" mean?



Two reasons to reduce the dimension.

1. To *visualize*: before choosing any model, you will want to plot your data, and your data will not be 2D.
2. To *preprocess*: fewer features means faster computations and less exposure to the curse of dimensionality (last session) for the next model.

This is only possible if the data does not use all the dimensions of its space.

The next slide explains what this means.

Why reduce the dimension?

High dimensional data is hard to handle (to visualize, to process, curse of dimensionality):

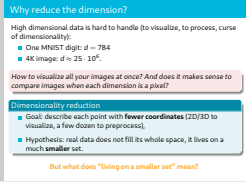
- One MNIST digit: $d = 784$
- 4K image: $d \approx 25 \cdot 10^6$.

How to visualize all your images at once? And does it makes sense to compare images when each dimension is a pixel?

Dimensionality reduction

- Goal: describe each point with **fewer coordinates** (2D/3D to visualize, a few dozen to preprocess),
- Hypothesis: real data does not fill its whole space, it lives on a much **smaller** set.

But what does "living on a smaller set" mean?



Two reasons to reduce the dimension.

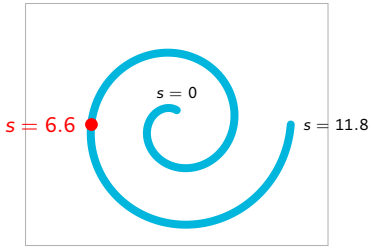
1. To *visualize*: before choosing any model, you will want to plot your data, and your data will not be 2D.
2. To *preprocess*: fewer features means faster computations and less exposure to the curse of dimensionality (last session) for the next model.

This is only possible if the data does not use all the dimensions of its space.

The next slide explains what this means.

Where does the data actually live?

A rolled up sheet (“swiss roll”)



Position in the plane: **2** numbers (x, y)

Position along the roll: **1** number (s)

One digit, rotated

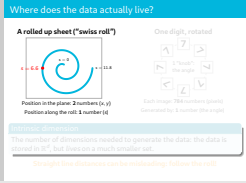
Each image: **784** numbers (pixels)

Generated by: **1** number (the angle)

Intrinsic dimension

The number of dimensions needed to generate the data: the data is *stored* in $\mathbb{R}^d$, but *lives* on a much smaller set.

Straight line distances can be misleading: follow the roll!



The key idea of dimensionality reduction, on two examples.

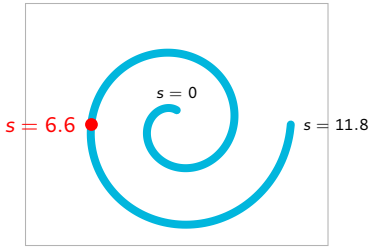
Left: the swiss roll, from last course. In the plane, each point needs two numbers (x, y). But the points cannot be anywhere: they stay on the roll, so one number s (the distance travelled along the roll from its start) locates them. The plane is where the data is *stored*; the roll is where it *lives*. Unrolling it is exactly what dimensionality reduction tries to do.

Right: each rotated image is a vector of 784 pixels, yet all of them are generated by a single “knob” (a button to be turned), the angle: in $\mathbb{R}^{784}$, they form a closed loop. Real data works the same way: photos of faces have millions of pixels, but a handful of “knobs” (or concepts, like head angle, lighting, expression) generate them. A random vector of $\mathbb{R}^{784}$ displayed as an image is pure noise, never a digit: real data occupies a tiny part of its space.

The last overlay is the trap: the two orange points are neighbours in a straight line (Euclidean distance), but far apart along the sheet, since one has to travel a whole turn of the roll to go from one to the other. (spoiler) PCA only knows straight directions and cannot unroll the swiss roll; nonlinear methods such as t-SNE try to.

Where does the data actually live?

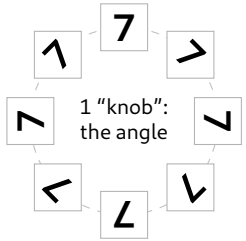
A rolled up sheet (“swiss roll”)



Position in the plane: **2** numbers (x, y)

Position along the roll: **1** number (s)

One digit, rotated



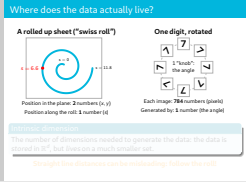
Each image: **784** numbers (pixels)

Generated by: **1** number (the angle)

Intrinsic dimension

The number of dimensions needed to generate the data: the data is *stored* in $\mathbb{R}^d$, but *lives* on a much smaller set.

Straight line distances can be misleading: follow the roll!



The key idea of dimensionality reduction, on two examples.

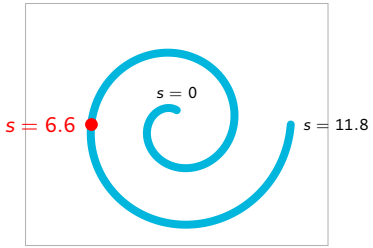
Left: the swiss roll, from last course. In the plane, each point needs two numbers (x, y). But the points cannot be anywhere: they stay on the roll, so one number s (the distance travelled along the roll from its start) locates them. The plane is where the data is *stored*; the roll is where it *lives*. Unrolling it is exactly what dimensionality reduction tries to do.

Right: each rotated image is a vector of 784 pixels, yet all of them are generated by a single “knob” (a button to be turned), the angle: in $\mathbb{R}^{784}$, they form a closed loop. Real data works the same way: photos of faces have millions of pixels, but a handful of “knobs” (or concepts, like head angle, lighting, expression) generate them. A random vector of $\mathbb{R}^{784}$ displayed as an image is pure noise, never a digit: real data occupies a tiny part of its space.

The last overlay is the trap: the two orange points are neighbours in a straight line (Euclidean distance), but far apart along the sheet, since one has to travel a whole turn of the roll to go from one to the other. (spoiler) PCA only knows straight directions and cannot unroll the swiss roll; nonlinear methods such as t-SNE try to.

Where does the data actually live?

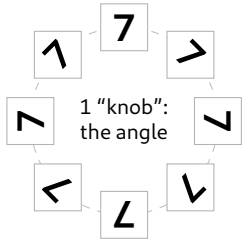
A rolled up sheet (“swiss roll”)



Position in the plane: **2** numbers (x, y)

Position along the roll: **1** number (s)

One digit, rotated



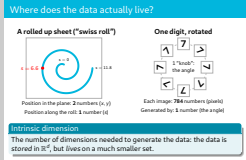
Each image: **784** numbers (pixels)

Generated by: **1** number (the angle)

Intrinsic dimension

The number of dimensions needed to generate the data: the data is *stored* in $\mathbb{R}^d$, but *lives* on a much smaller set.

Straight line distances can be misleading: follow the roll!



The key idea of dimensionality reduction, on two examples.

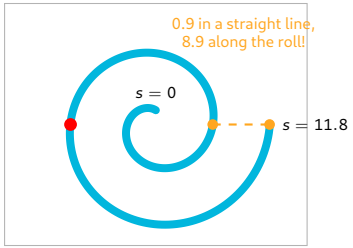
Left: the swiss roll, from last course. In the plane, each point needs two numbers (x, y). But the points cannot be anywhere: they stay on the roll, so one number s (the distance travelled along the roll from its start) locates them. The plane is where the data is *stored*; the roll is where it *lives*. Unrolling it is exactly what dimensionality reduction tries to do.

Right: each rotated image is a vector of 784 pixels, yet all of them are generated by a single “knob” (a button to be turned), the angle: in $\mathbb{R}^{784}$, they form a closed loop. Real data works the same way: photos of faces have millions of pixels, but a handful of “knobs” (or concepts, like head angle, lighting, expression) generate them. A random vector of $\mathbb{R}^{784}$ displayed as an image is pure noise, never a digit: real data occupies a tiny part of its space.

The last overlay is the trap: the two orange points are neighbours in a straight line (Euclidean distance), but far apart along the sheet, since one has to travel a whole turn of the roll to go from one to the other. (spoiler) PCA only knows straight directions and cannot unroll the swiss roll; nonlinear methods such as t-SNE try to.

Where does the data actually live?

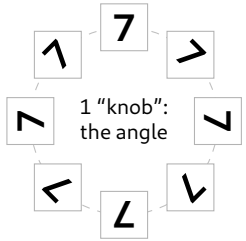
A rolled up sheet ("swiss roll")



Position in the plane: **2** numbers (x, y)

Position along the roll: **1** number (s)

One digit, rotated



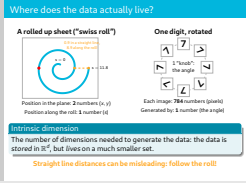
Each image: **784** numbers (pixels)

Generated by: **1** number (the angle)

Intrinsic dimension

The number of dimensions needed to generate the data: the data is *stored* in $\mathbb{R}^d$, but *lives* on a much smaller set.

Straight line distances can be misleading: follow the roll!



The key idea of dimensionality reduction, on two examples.

Left: the swiss roll, from last course. In the plane, each point needs two numbers (x, y). But the points cannot be anywhere: they stay on the roll, so one number s (the distance travelled along the roll from its start) locates them. The plane is where the data is *stored*; the roll is where it *lives*. Unrolling it is exactly what dimensionality reduction tries to do.

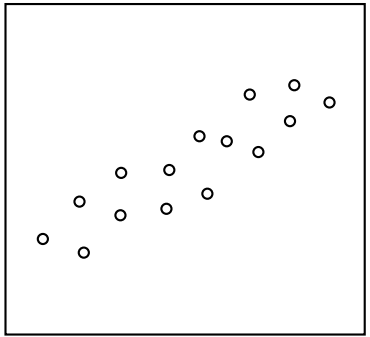
Right: each rotated image is a vector of 784 pixels, yet all of them are generated by a single "knob" (a button to be turned), the angle: in $\mathbb{R}^{784}$, they form a closed loop. Real data works the same way: photos of faces have millions of pixels, but a handful of "knobs" (or concepts, like head angle, lighting, expression) generate them. A random vector of $\mathbb{R}^{784}$ displayed as an image is pure noise, never a digit: real data occupies a tiny part of its space.

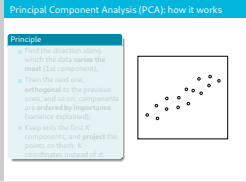
The last overlay is the trap: the two orange points are neighbours in a straight line (Euclidean distance), but far apart along the sheet, since one has to travel a whole turn of the roll to go from one to the other. (spoiler) PCA only knows straight directions and cannot unroll the swiss roll; nonlinear methods such as t-SNE try to.

Principal Component Analysis (PCA): how it works

Principle

- Find the direction along which the data **varies the most** (1st component),
- Then the next one, **orthogonal** to the previous ones, and so on: components are **ordered by importance** (variance explained),
- Keep only the first K components, and **project** the points on them: K coordinates instead of d.





PCA is the simplest and by far the most used dimensionality reduction method. Follow the animation: the first component is the direction in which the cloud is the most stretched; the second one is orthogonal to it and captures what remains. In dimension d there are d such directions, ordered by the share of variance they explain.

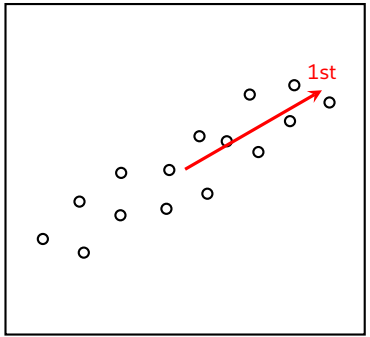
Reducing the dimension then simply means keeping the first K components and projecting each point on them (the orange points): each point is now described by K numbers. What is lost is the distance between the points and their projections (the dashed lines): PCA chooses the directions that make this loss as small as possible.

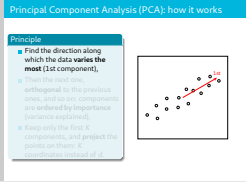
For the curious: the components are the eigenvectors of the covariance matrix of the centered data, and the variance they explain is given by the corresponding eigenvalues.

Principal Component Analysis (PCA): how it works

Principle

- Find the direction along which the data **varies the most** (1st component),
- Then the next one, **orthogonal** to the previous ones, and so on: components are **ordered by importance** (variance explained),
- Keep only the first K components, and **project** the points on them: K coordinates instead of d.





PCA is the simplest and by far the most used dimensionality reduction method. Follow the animation: the first component is the direction in which the cloud is the most stretched; the second one is orthogonal to it and captures what remains. In dimension d there are d such directions, ordered by the share of variance they explain.

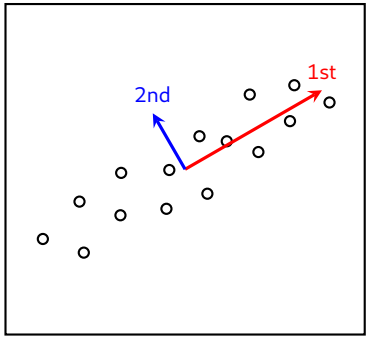
Reducing the dimension then simply means keeping the first K components and projecting each point on them (the orange points): each point is now described by K numbers. What is lost is the distance between the points and their projections (the dashed lines): PCA chooses the directions that make this loss as small as possible.

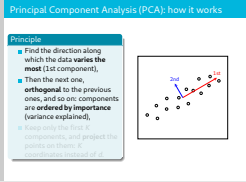
For the curious: the components are the eigenvectors of the covariance matrix of the centered data, and the variance they explain is given by the corresponding eigenvalues.

Principal Component Analysis (PCA): how it works

Principle

- Find the direction along which the data **varies the most** (1st component),
- Then the next one, **orthogonal** to the previous ones, and so on: components are **ordered by importance** (variance explained),
- Keep only the first K components, and **project** the points on them: K coordinates instead of d.





PCA is the simplest and by far the most used dimensionality reduction method. Follow the animation: the first component is the direction in which the cloud is the most stretched; the second one is orthogonal to it and captures what remains. In dimension d there are d such directions, ordered by the share of variance they explain.

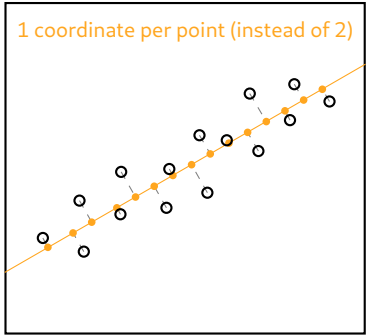
Reducing the dimension then simply means keeping the first K components and projecting each point on them (the orange points): each point is now described by K numbers. What is lost is the distance between the points and their projections (the dashed lines): PCA chooses the directions that make this loss as small as possible.

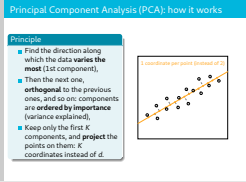
For the curious: the components are the eigenvectors of the covariance matrix of the centered data, and the variance they explain is given by the corresponding eigenvalues.

Principal Component Analysis (PCA): how it works

Principle

- Find the direction along which the data **varies the most** (1st component),
- Then the next one, **orthogonal** to the previous ones, and so on: components are **ordered by importance** (variance explained),
- Keep only the first K components, and **project** the points on them: K coordinates instead of d.





PCA is the simplest and by far the most used dimensionality reduction method. Follow the animation: the first component is the direction in which the cloud is the most stretched; the second one is orthogonal to it and captures what remains. In dimension d there are d such directions, ordered by the share of variance they explain.

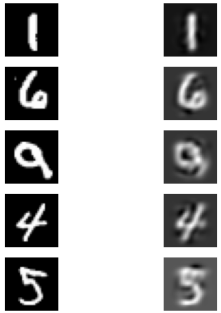
Reducing the dimension then simply means keeping the first K components and projecting each point on them (the orange points): each point is now described by K numbers. What is lost is the distance between the points and their projections (the dashed lines): PCA chooses the directions that make this loss as small as possible.

For the curious: the components are the eigenvectors of the covariance matrix of the centered data, and the variance they explain is given by the corresponding eigenvalues.

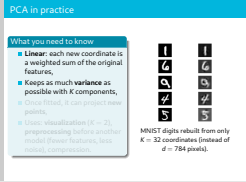
PCA in practice

What you need to know

- **Linear**: each new coordinate is a weighted sum of the original features,
- Keeps as much **variance** as possible with K components,
- Once fitted, it can project **new points**,
- Uses: **visualization** ($K = 2$), **preprocessing** before another model (fewer features, less noise), compression.



MNIST digits rebuilt from only $K = 32$ coordinates (instead of $d = 784$ pixels).



On MNIST: 32 coordinates instead of 784 pixels, and the digits rebuilt from them remain readable: roughly a factor 24, obtained without any label. The slight blur is the visible trace of the discarded variance. As a preprocessing step, feeding these 32 coordinates to a classifier instead of the 784 pixels makes it faster and mitigates the curse of dimensionality.

Two consequences of the ordering. The components are nested: the first two of a $K = 32$ reduction are the same as those of a $K = 2$ one, which is why PCA is one of the default tools for 2D visualization. And `explained_variance_ratio_` tells you how much of the data you keep; its cumulative sum thus provides a way to choose K.

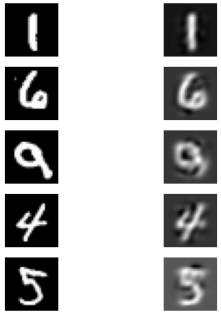
Since PCA is a fixed linear transform, `transform` can be applied to new points (NB: fit it on the training set only when it precedes a supervised model, to avoid data leakage).

Warning: PCA is variance-based, so run it on scaled/normalized features, otherwise the feature with the largest units wins.

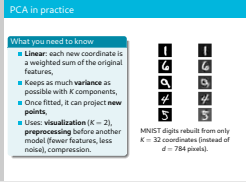
PCA in practice

What you need to know

- **Linear**: each new coordinate is a weighted sum of the original features,
- Keeps as much **variance** as possible with K components,
- Once fitted, it can project **new points**,
- Uses: **visualization** ($K = 2$), **preprocessing** before another model (fewer features, less noise), compression.



MNIST digits rebuilt from only $K = 32$ coordinates (instead of $d = 784$ pixels).



On MNIST: 32 coordinates instead of 784 pixels, and the digits rebuilt from them remain readable: roughly a factor 24, obtained without any label. The slight blur is the visible trace of the discarded variance. As a preprocessing step, feeding these 32 coordinates to a classifier instead of the 784 pixels makes it faster and mitigates the curse of dimensionality.

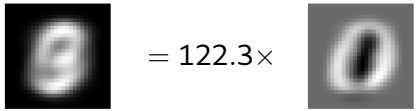
Two consequences of the ordering. The components are nested: the first two of a $K = 32$ reduction are the same as those of a $K = 2$ one, which is why PCA is one of the default tools for 2D visualization. And `explained_variance_ratio_` tells you how much of the data you keep; its cumulative sum thus provides a way to choose K.

Since PCA is a fixed linear transform, `transform` can be applied to new points (NB: fit it on the training set only when it precedes a supervised model, to avoid data leakage).

Warning: PCA is variance-based, so run it on scaled/normalized features, otherwise the feature with the largest units wins.

PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time



PCA on MNIST: step by step
Rebuilding one digit, adding one component at a time



One digit, rebuilt by adding one component at a time: the image on the left is the sum of the components on the right, each multiplied by the coordinate of this digit along it (the numbers). Watch how few components are needed before the digit becomes recognisable.

The components themselves are vectors of the same dimension as the images, so they can be displayed as images: they look like “ghost digits”, the first ones capturing the average shape and the main strokes, the last ones the fine details and the noise.

PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time



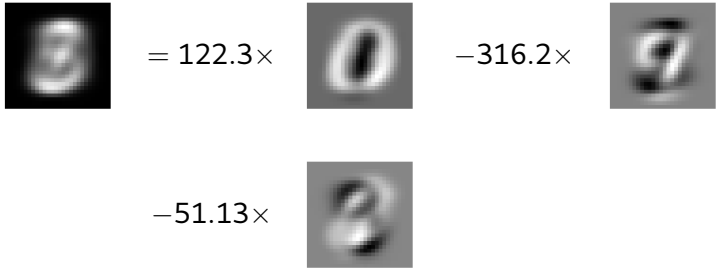
PCA on MNIST: step by step
Rebuilding one digit, adding one component at a time

One digit, rebuilt by adding one component at a time: the image on the left is the sum of the components on the right, each multiplied by the coordinate of this digit along it (the numbers). Watch how few components are needed before the digit becomes recognisable.

The components themselves are vectors of the same dimension as the images, so they can be displayed as images: they look like “ghost digits”, the first ones capturing the average shape and the main strokes, the last ones the fine details and the noise.

PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time



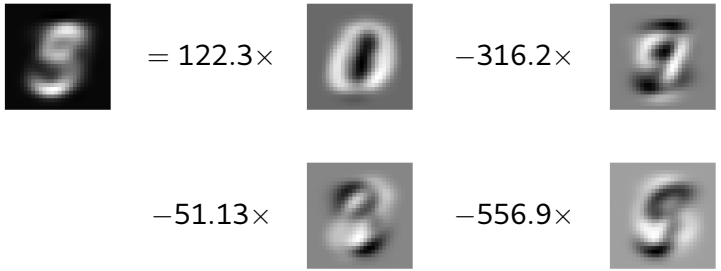
PCA on MNIST: step by step
Rebuilding one digit, adding one component at a time

One digit, rebuilt by adding one component at a time: the image on the left is the sum of the components on the right, each multiplied by the coordinate of this digit along it (the numbers). Watch how few components are needed before the digit becomes recognisable.

The components themselves are vectors of the same dimension as the images, so they can be displayed as images: they look like “ghost digits”, the first ones capturing the average shape and the main strokes, the last ones the fine details and the noise.

PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time



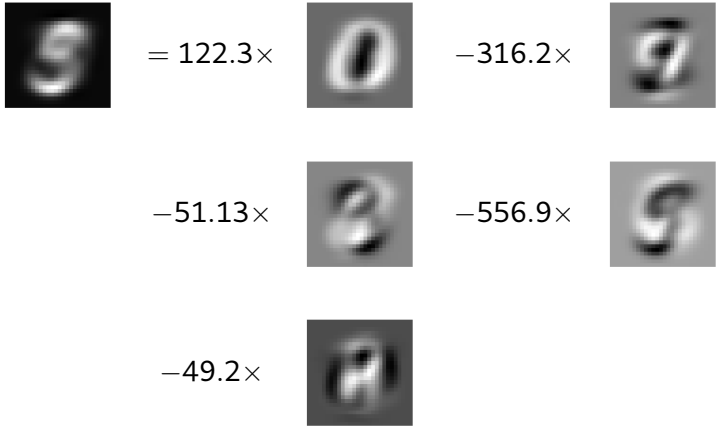
PCA on MNIST: step by step
Rebuilding one digit, adding one component at a time

One digit, rebuilt by adding one component at a time: the image on the left is the sum of the components on the right, each multiplied by the coordinate of this digit along it (the numbers). Watch how few components are needed before the digit becomes recognisable.

The components themselves are vectors of the same dimension as the images, so they can be displayed as images: they look like “ghost digits”, the first ones capturing the average shape and the main strokes, the last ones the fine details and the noise.

PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time



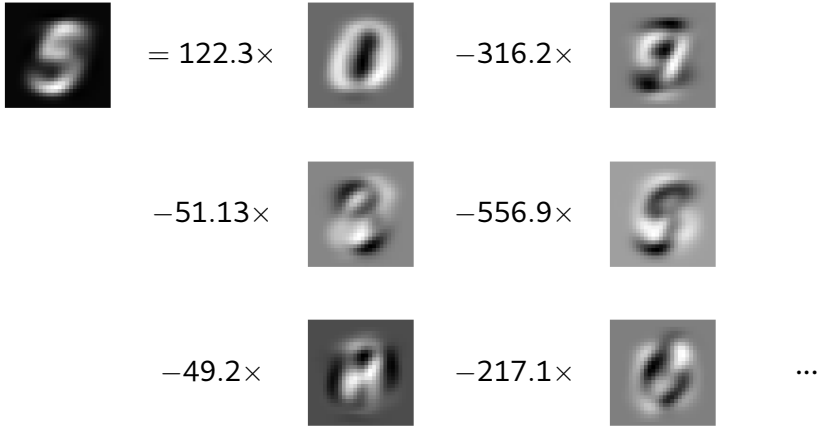
PCA on MNIST: step by step
Rebuilding one digit, adding one component at a time

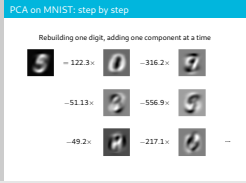
One digit, rebuilt by adding one component at a time: the image on the left is the sum of the components on the right, each multiplied by the coordinate of this digit along it (the numbers). Watch how few components are needed before the digit becomes recognisable.

The components themselves are vectors of the same dimension as the images, so they can be displayed as images: they look like “ghost digits”, the first ones capturing the average shape and the main strokes, the last ones the fine details and the noise.

PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time





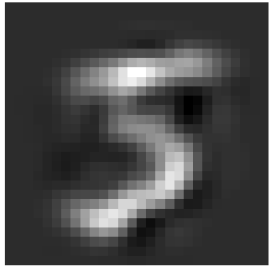
One digit, rebuilt by adding one component at a time: the image on the left is the sum of the components on the right, each multiplied by the coordinate of this digit along it (the numbers). Watch how few components are needed before the digit becomes recognisable.

The components themselves are vectors of the same dimension as the images, so they can be displayed as images: they look like “ghost digits”, the first ones capturing the average shape and the main strokes, the last ones the fine details and the noise.

PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time

Reconstruction with all 32 components:



PCA on MNIST: step by step
Rebuilding one digit, adding one component at a time
Reconstruction with all 32 components:



One digit, rebuilt by adding one component at a time: the image on the left is the sum of the components on the right, each multiplied by the coordinate of this digit along it (the numbers). Watch how few components are needed before the digit becomes recognisable.

The components themselves are vectors of the same dimension as the images, so they can be displayed as images: they look like “ghost digits”, the first ones capturing the average shape and the main strokes, the last ones the fine details and the noise.

t-SNE: a nonlinear method for visualization

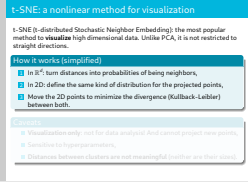
t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data. Unlike PCA, it is not restricted to straight directions.

How it works (simplified)

1. In $\mathbb{R}^d$: turn distances into probabilities of being neighbors,
2. In 2D: define the same kind of distribution for the projected points,
3. Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

Caveats

- **Visualization only**: not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).



We stay at the level of intuition here; the details of the Kullback-Leibler divergence are out of scope. The take-home message: t-SNE only tries to keep neighbours together, and does not care about what happens far away. This is what allows it to follow curved surfaces such as the swiss roll.

The caveats are what you actually need to keep in mind for the lab and project.!The last one is the most frequently violated: a t-SNE plot is not a map, so commenting on the distance between two blobs, or on the size of a blob, is meaningless: both are artefacts of the optimization! A t-SNE plot answers “are there groups?”, not “how far apart are they?”.

UMAP is another example, and used in the same way as t-SNE. Nonetheless, it is usually faster and preserves a bit more of the global structure.

t-SNE: a nonlinear method for visualization

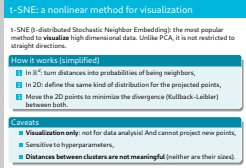
t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data. Unlike PCA, it is not restricted to straight directions.

How it works (simplified)

1. In $\mathbb{R}^d$: turn distances into probabilities of being neighbors,
2. In 2D: define the same kind of distribution for the projected points,
3. Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

Caveats

- **Visualization only**: not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

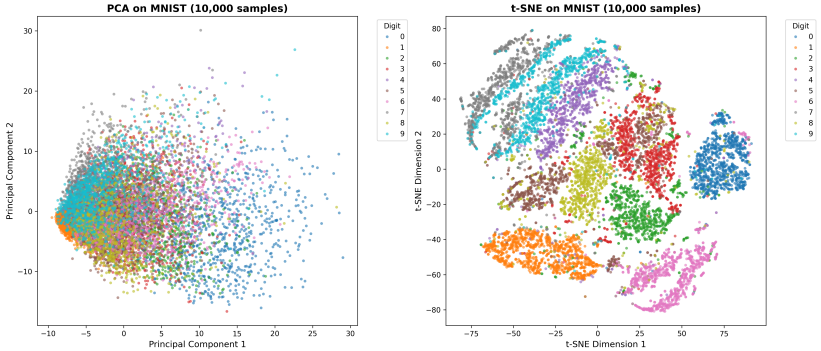


We stay at the level of intuition here; the details of the Kullback-Leibler divergence are out of scope. The take-home message: t-SNE only tries to keep neighbours together, and does not care about what happens far away. This is what allows it to follow curved surfaces such as the swiss roll.

The caveats are what you actually need to keep in mind for the lab and project.!The last one is the most frequently violated: a t-SNE plot is not a map, so commenting on the distance between two blobs, or on the size of a blob, is meaningless: both are artefacts of the optimization! A t-SNE plot answers "are there groups?", not "how far apart are they?".

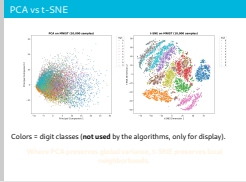
UMAP is another example, and used in the same way as t-SNE. Nonetheless, it is usually faster and preserves a bit more of the global structure.

PCA vs t-SNE



Colors = digit classes (**not used** by the algorithms, only for display).

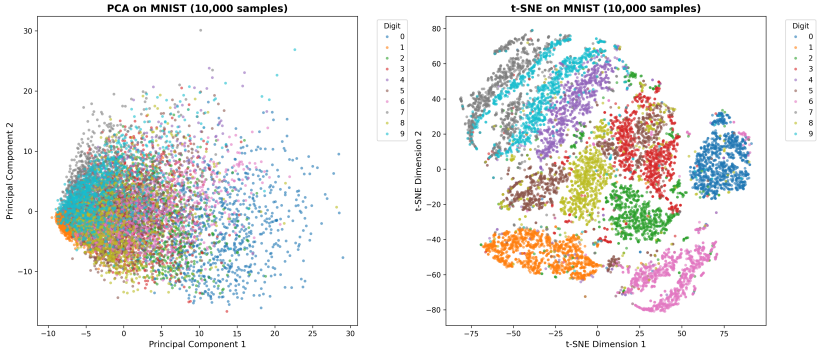
Where PCA preserves global variance, t-SNE preserves local neighborhoods.



The same digits projected in 2D by both methods. PCA, being a linear projection, superimposes classes; t-SNE separates them into visible groups.
Two remarks: The colours are labels used only for *display*: both algorithms ran without them. Doing exactly this is the cheapest sanity check of an unsupervised pipeline. And do not over-trust t-SNE: clean groups do not guarantee that a classifier will separate the classes, and PCA remains the tool of choice when the projection must be reused, since it is a real linear transform applicable to new data.

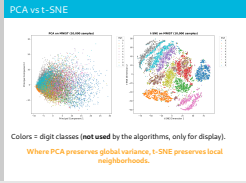
The two methods are complementary rather than competing. A common practice: run PCA first (down to ≈ 50 dimensions) to denoise and speed things up, then t-SNE on the result for the 2D picture.

PCA vs t-SNE



Colors = digit classes (**not used** by the algorithms, only for display).

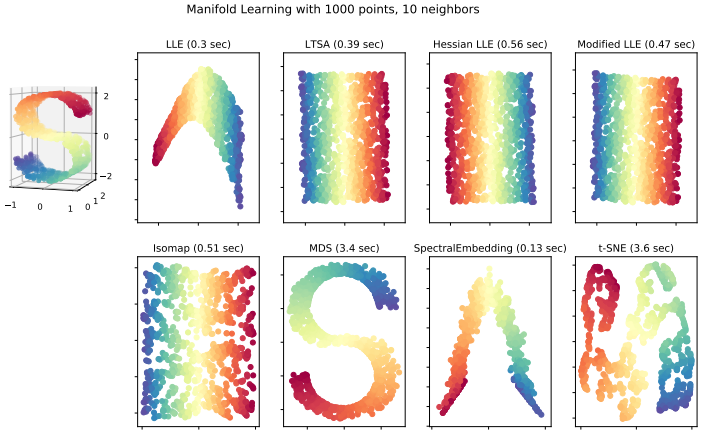
Where PCA preserves global variance, t-SNE preserves local neighborhoods.



The same digits projected in 2D by both methods. PCA, being a linear projection, superimposes classes; t-SNE separates them into visible groups.
Two remarks: The colours are labels used only for *display*: both algorithms ran without them. Doing exactly this is the cheapest sanity check of an unsupervised pipeline. And do not over-trust t-SNE: clean groups do not guarantee that a classifier will separate the classes, and PCA remains the tool of choice when the projection must be reused, since it is a real linear transform applicable to new data.

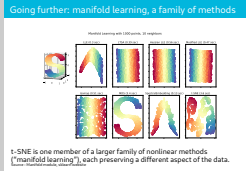
The two methods are complementary rather than competing. A common practice: run PCA first (down to ≈ 50 dimensions) to denoise and speed things up, then t-SNE on the result for the 2D picture.

Going further: manifold learning, a family of methods



t-SNE is one member of a larger family of nonlinear methods ("manifold learning"), each preserving a different aspect of the data.

Source : Manifold module, sklearn website



An opening, not something to learn by heart; we will not detail how each of these methods works. What matters is the range of them: they all take the same input (points in $\mathbb{R}^d$) and produce the same output (points in 2D), but they disagree on what must be preserved (geodesic distances, local neighbourhoods, local linear reconstructions, …), hence very different pictures of the same dataset.

The original data is the swiss roll shown in the top-left corner of the figure. In practice, t-SNE (or UMAP) is the most common default for visualization; if you are curious, try one or two others in the lab and compare the pictures.

Plan

1 Clustering

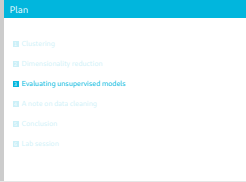
2 Dimensionality reduction

3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session

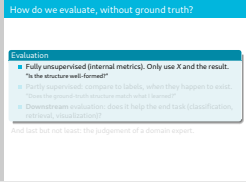


How do we evaluate, without ground truth?

Evaluation

- Fully unsupervised (internal metrics). Only use X and the result.
  "Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.
  "Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.



Last session, evaluation was conceptually simple: there was a right answer, and we counted the mistakes. In unsupervised, we have to assume that this is not possible (even if one can apply unsupervised methods on annotated datasets). When no ground truth is available, you still have to defend your choices.

All metrics are not always possible. Internal metrics are always available, but they only measure a geometric property that we *decided* was desirable: a dataset with no cluster structure will still hand you a "best" K. Metrics based on labels are more convincing, but they need labels (in practice they are used on a subset, or on a public dataset, to validate a pipeline before applying it where no label exists). The downstream evaluation is the most honest one in a real project: if the result improves the next model, or helps someone take a decision, it is good.

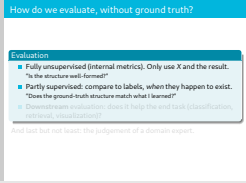
The take-away: an unsupervised result is never "correct", it is *useful* or not.

How do we evaluate, without ground truth?

Evaluation

- Fully unsupervised (internal metrics). Only use X and the result.
  "Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.
  "Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.



Last session, evaluation was conceptually simple: there was a right answer, and we counted the mistakes. In unsupervised, we have to assume that this is not possible (even if one can apply unsupervised methods on annotated datasets). When no ground truth is available, you still have to defend your choices.

All metrics are not always possible. Internal metrics are always available, but they only measure a geometric property that we *decided* was desirable: a dataset with no cluster structure will still hand you a "best" K. Metrics based on labels are more convincing, but they need labels (in practice they are used on a subset, or on a public dataset, to validate a pipeline before applying it where no label exists). The downstream evaluation is the most honest one in a real project: if the result improves the next model, or helps someone take a decision, it is good.

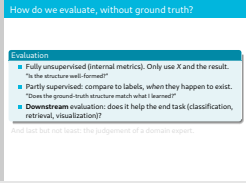
The take-away: an unsupervised result is never "correct", it is *useful* or not.

How do we evaluate, without ground truth?

Evaluation

- Fully unsupervised (internal metrics). Only use X and the result.
  "Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.
  "Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.



Last session, evaluation was conceptually simple: there was a right answer, and we counted the mistakes. In unsupervised, we have to assume that this is not possible (even if one can apply unsupervised methods on annotated datasets). When no ground truth is available, you still have to defend your choices.

All metrics are not always possible. Internal metrics are always available, but they only measure a geometric property that we *decided* was desirable: a dataset with no cluster structure will still hand you a "best" K. Metrics based on labels are more convincing, but they need labels (in practice they are used on a subset, or on a public dataset, to validate a pipeline before applying it where no label exists). The downstream evaluation is the most honest one in a real project: if the result improves the next model, or helps someone take a decision, it is good.

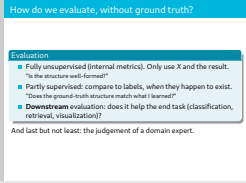
The take-away: an unsupervised result is never "correct", it is *useful* or not.

How do we evaluate, without ground truth?

Evaluation

- Fully unsupervised (internal metrics). Only use X and the result.
  "Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.
  "Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.



Last session, evaluation was conceptually simple: there was a right answer, and we counted the mistakes. In unsupervised, we have to assume that this is not possible (even if one can apply unsupervised methods on annotated datasets). When no ground truth is available, you still have to defend your choices.

All metrics are not always possible. Internal metrics are always available, but they only measure a geometric property that we *decided* was desirable: a dataset with no cluster structure will still hand you a "best" K. Metrics based on labels are more convincing, but they need labels (in practice they are used on a subset, or on a public dataset, to validate a pipeline before applying it where no label exists). The downstream evaluation is the most honest one in a real project: if the result improves the next model, or helps someone take a decision, it is good.

The take-away: an unsupervised result is never "correct", it is *useful* or not.

Fully unsupervised clustering metrics

Inertia

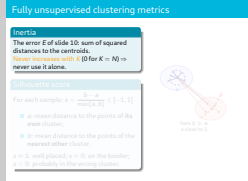
The error E of slide 10: sum of squared distances to the centroids.
Never increases with K (0 for $K = N$) $\Rightarrow$ never use it alone.

Silhouette score

For each sample: $s = \dfrac{b - a}{\max(a, b)} \in [-1, 1]$

- a: mean distance to the points of **its own** cluster,
- b: mean distance to the points of the **nearest other** cluster.

$s \approx 1$: well placed; $s \approx 0$: on the border; $s < 0$: probably in the wrong cluster.



The silhouette is the metric you will use most in practice, so read the formula on the drawing: a measures how well the point sits in its own cluster, b how far it is from the competition, and the score compares the two. The black dot is **one sample among the others**, not a centroid: both a and b are averages of distances to individual points, and the two dashed arcs are drawn exactly at those average distances.

Three properties. It is defined *per sample*, so you can plot the whole distribution and see which clusters are badly formed, instead of a single average. It is bounded in $[-1, 1]$, so unlike inertia it can be compared across values of K (which is what makes it usable to choose K). And it costs $O(N^2)$ distances, so on a large dataset compute it on a subsample.

Its bias is worth knowing: like inertia, it rewards compact convex clusters, so it will prefer a K-means solution to a DBSCAN one on non-convex data, even when DBSCAN is visibly right. A metric always encodes a definition of what a good cluster is.

Fully unsupervised clustering metrics

Inertia

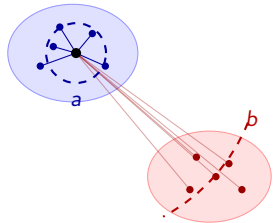
The error E of slide 10: sum of squared distances to the centroids.
Never increases with K (0 for $K = N$) $\Rightarrow$ never use it alone.

Silhouette score

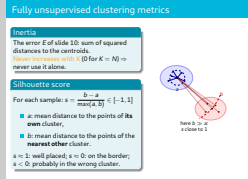
For each sample: $s = \dfrac{b - a}{\max(a, b)} \in [-1, 1]$

- a: mean distance to the points of **its own** cluster,
- b: mean distance to the points of the **nearest other** cluster.

$s \approx 1$: well placed; $s \approx 0$: on the border;
$s < 0$: probably in the wrong cluster.



here $b \gg a$:
s close to 1

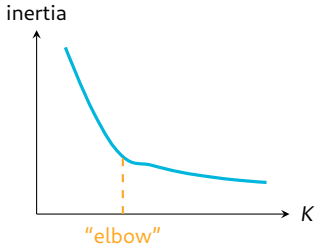


The silhouette is the metric you will use most in practice, so read the formula on the drawing: a measures how well the point sits in its own cluster, b how far it is from the competition, and the score compares the two. The black dot is **one sample among the others**, not a centroid: both a and b are averages of distances to individual points, and the two dashed arcs are drawn exactly at those average distances.

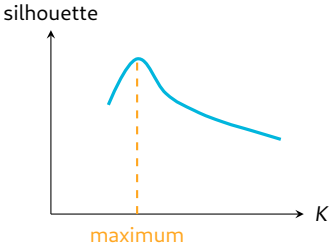
Three properties. It is defined *per sample*, so you can plot the whole distribution and see which clusters are badly formed, instead of a single average. It is bounded in $[-1, 1]$, so unlike inertia it can be compared across values of K (which is what makes it usable to choose K). And it costs $O(N^2)$ distances, so on a large dataset compute it on a subsample.

Its bias is worth knowing: like inertia, it rewards compact convex clusters, so it will prefer a K-means solution to a DBSCAN one on non-convex data, even when DBSCAN is visibly right. A metric always encodes a definition of what a good cluster is.

Back to choosing K: two heuristics



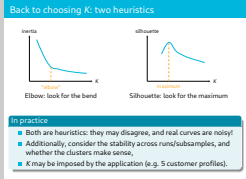
Elbow: look for the bend



Silhouette: look for the maximum

In practice

- Both are heuristics: they may disagree, and real curves are noisy!
- Additionally, consider the stability across runs/subsamples, and whether the clusters make sense,
- K may be imposed by the application (e.g. 5 customer profiles).



The recipe for the lab: compute both curves, look at them, decide, and *justify*. The elbow is the practical answer to the trade-off of slide 14: since the inertia always decreases with K, we look for the value after which adding clusters barely reduces it, i.e. where gaining compression stops costing much error. These two curves are idealised. On real data the elbow is often invisible, and the silhouette often decreases from $K = 2$ onwards (a legitimate result that indicates "there is no obvious cluster structure"). You are allowed to conclude that, rather than forcing a value.

The last bullet is what professionals use most: the number of clusters is frequently a constraint of the application (a marketing team handles 5 segments, not 43). The metrics then serve to check that the imposed K is not absurd. Finally, since K-means depends on its initialization, run it several times and check that the clusters are stable: an unstable clustering is a strong sign that the structure is not there.

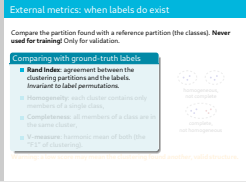
External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

Comparing with ground-truth labels

- **Rand Index**: agreement between the clustering partitions and the labels. *Invariant to label permutations.*
- **Homogeneity**: each cluster contains only members of a single class,
- **Completeness**: all members of a class are in the same cluster,
- **V-measure**: harmonic mean of both (the "F1" of clustering).

Warning: a low score may mean the clustering found *another*, valid structure.



Rand index ignores permutation, because identifying the meaning of the clusters is not part of the clustering process. Only the grouping matters, not the assignment of clusters with labels (which is why accuracy cannot be used directly here).

The two drawings make homogeneity and completeness concrete. Splitting one class into two perfect halves is homogeneous but not complete; merging two classes into one cluster is complete but not homogeneous. Watch the degenerate cases: $K = N$ is perfectly homogeneous, $K = 1$ perfectly complete. Hence the V-measure, which combines both. The similarity with F1-score is not accidental, it kind of is the same idea of balancing over and under-prediction.

An example for the warning: clustering photographs of people may group them by lighting conditions rather than by identity. Compared to identity labels the score is terrible, although the algorithm found a real and consistent structure. A metric measures agreement with *one particular* question.

External metrics: when labels do exist

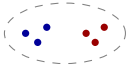
Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

Comparing with ground-truth labels

- **Rand Index**: agreement between the clustering partitions and the labels. *Invariant to label permutations.*
- **Homogeneity**: each cluster contains only members of a single class,
- **Completeness**: all members of a class are in the same cluster,
- **V-measure**: harmonic mean of both (the "F1" of clustering).

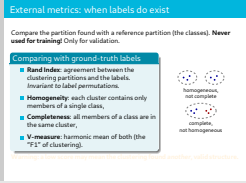


homogeneous, not complete



complete, not homogeneous

Warning: a low score may mean the clustering found *another*, valid structure.



Rand index ignores permutation, because identifying the meaning of the clusters is not part of the clustering process. Only the grouping matters, not the assignment of clusters with labels (which is why accuracy cannot be used directly here).

The two drawings make homogeneity and completeness concrete. Splitting one class into two perfect halves is homogeneous but not complete; merging two classes into one cluster is complete but not homogeneous. Watch the degenerate cases: $K = N$ is perfectly homogeneous, $K = 1$ perfectly complete. Hence the V-measure, which combines both. The similarity with F1-score is not accidental, it kind of is the same idea of balancing over and under-prediction.

An example for the warning: clustering photographs of people may group them by lighting conditions rather than by identity. Compared to identity labels the score is terrible, although the algorithm found a real and consistent structure. A metric measures agreement with *one particular* question.

External metrics: when labels do exist

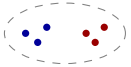
Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

Comparing with ground-truth labels

- **Rand Index**: agreement between the clustering partitions and the labels. *Invariant to label permutations.*
- **Homogeneity**: each cluster contains only members of a single class,
- **Completeness**: all members of a class are in the same cluster,
- **V-measure**: harmonic mean of both (the "F1" of clustering).

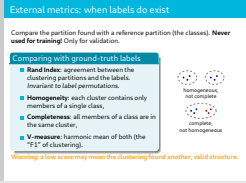


homogeneous, not complete



complete, not homogeneous

Warning: a low score may mean the clustering found *another*, valid structure.



Rand index ignores permutation, because identifying the meaning of the clusters is not part of the clustering process. Only the grouping matters, not the assignment of clusters with labels (which is why accuracy cannot be used directly here).

The two drawings make homogeneity and completeness concrete. Splitting one class into two perfect halves is homogeneous but not complete; merging two classes into one cluster is complete but not homogeneous. Watch the degenerate cases: $K = N$ is perfectly homogeneous, $K = 1$ perfectly complete. Hence the V-measure, which combines both. The similarity with F1-score is not accidental, it kind of is the same idea of balancing over and under-prediction.

An example for the warning: clustering photographs of people may group them by lighting conditions rather than by identity. Compared to identity labels the score is terrible, although the algorithm found a real and consistent structure. A metric measures agreement with *one particular* question.

Evaluating dimensionality reduction

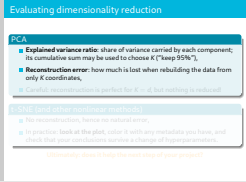
PCA

- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose K ("keep 95%"),
- **Reconstruction error**: how much is lost when rebuilding the data from only K coordinates,
- Careful: reconstruction is perfect for $K = d$, but nothing is reduced!

t-SNE (and other nonlinear methods)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?



Each method gets its own kind of evaluation, and none of them gives a "score out of 10".
For PCA, the cumulative explained variance is one of the most defensible way to justify a value of K. It behaves like the elbow: the curve rises and saturates, and you pick the point where adding components stops paying.
For t-SNE the evaluation is essentially visual, which is exactly why you need to keep its caveats in mind. The robustness check is your only real protection: if the groups you see disappear when you change the perplexity, they were an artefact.

The closing sentence is the message of the whole session. In your project, an unsupervised step is justified by what it enables: a cleaner input for the next model, a plot that convinced you of something, a segmentation someone could act on.

Evaluating dimensionality reduction

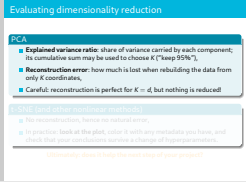
PCA

- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose K (“keep 95%”),
- **Reconstruction error**: how much is lost when rebuilding the data from only K coordinates,
- Careful: reconstruction is perfect for $K = d$, but nothing is reduced!

t-SNE (and other nonlinear methods)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?



Each method gets its own kind of evaluation, and none of them gives a “score out of 10”.
For PCA, the cumulative explained variance is one of the most defensible way to justify a value of K. It behaves like the elbow: the curve rises and saturates, and you pick the point where adding components stops paying.
For t-SNE the evaluation is essentially visual, which is exactly why you need to keep its caveats in mind. The robustness check is your only real protection: if the groups you see disappear when you change the perplexity, they were an artefact.

The closing sentence is the message of the whole session. In your project, an unsupervised step is justified by what it enables: a cleaner input for the next model, a plot that convinced you of something, a segmentation someone could act on.

Evaluating dimensionality reduction

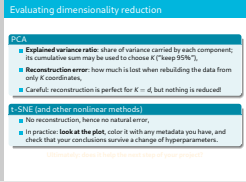
PCA

- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose K ("keep 95%"),
- **Reconstruction error**: how much is lost when rebuilding the data from only K coordinates,
- Careful: reconstruction is perfect for $K = d$, but nothing is reduced!

t-SNE (and other nonlinear methods)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?



Each method gets its own kind of evaluation, and none of them gives a "score out of 10".
For PCA, the cumulative explained variance is one of the most defensible way to justify a value of K. It behaves like the elbow: the curve rises and saturates, and you pick the point where adding components stops paying.
For t-SNE the evaluation is essentially visual, which is exactly why you need to keep its caveats in mind. The robustness check is your only real protection: if the groups you see disappear when you change the perplexity, they were an artefact.

The closing sentence is the message of the whole session. In your project, an unsupervised step is justified by what it enables: a cleaner input for the next model, a plot that convinced you of something, a segmentation someone could act on.

Evaluating dimensionality reduction

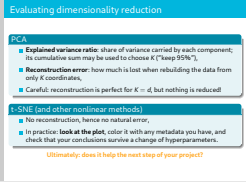
PCA

- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose K (“keep 95%”),
- **Reconstruction error**: how much is lost when rebuilding the data from only K coordinates,
- Careful: reconstruction is perfect for $K = d$, but nothing is reduced!

t-SNE (and other nonlinear methods)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?



Each method gets its own kind of evaluation, and none of them gives a “score out of 10”.
For PCA, the cumulative explained variance is one of the most defensible way to justify a value of K. It behaves like the elbow: the curve rises and saturates, and you pick the point where adding components stops paying.
For t-SNE the evaluation is essentially visual, which is exactly why you need to keep its caveats in mind. The robustness check is your only real protection: if the groups you see disappear when you change the perplexity, they were an artefact.

The closing sentence is the message of the whole session. In your project, an unsupervised step is justified by what it enables: a cleaner input for the next model, a plot that convinced you of something, a segmentation someone could act on.

Plan

1 Clustering

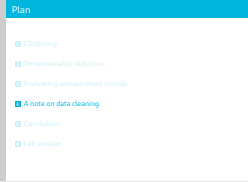
2 Dimensionality reduction

3 Evaluating unsupervised models

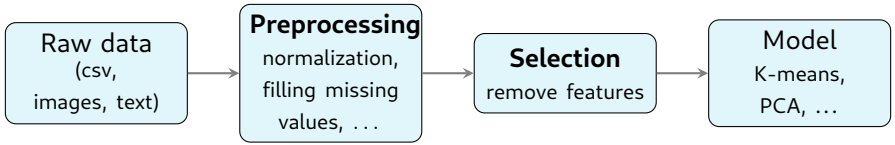
4 A note on data cleaning

5 Conclusion

6 Lab session



Features in standard Machine Learning pipelines



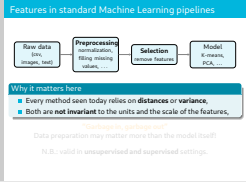
Why it matters here

- Every method seen today relies on **distances** or **variance**,
- Both are **not invariant** to the units and the scale of the features,

"Garbage in, garbage out"

Data preparation may matter more than the model itself!

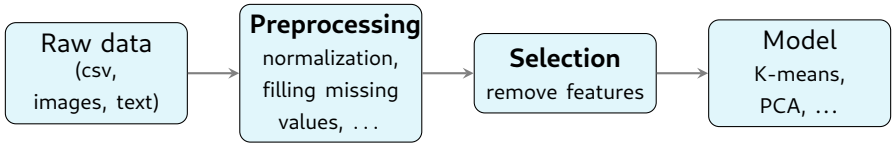
N.B.: valid in **unsupervised and supervised** settings.



The "boring but decisive" part of the pipeline (at least for standard Machine Learning).

Here is why it closes the session, after the methods and their evaluation: K-means compares distances, PCA maximises variance, t-SNE compares neighbourhoods. All three change completely if one feature is expressed in euros and another in thousands of euros. Unlike some supervised methods (e.g., decision tree), these methods cannot compensate for a bad scaling.

Features in standard Machine Learning pipelines



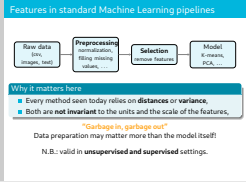
Why it matters here

- Every method seen today relies on **distances** or **variance**,
- Both are **not invariant** to the units and the scale of the features,

"Garbage in, garbage out"

Data preparation may matter more than the model itself!

N.B.: valid in **unsupervised and supervised** settings.



The "boring but decisive" part of the pipeline (at least for standard Machine Learning).

Here is why it closes the session, after the methods and their evaluation: K-means compares distances, PCA maximises variance, t-SNE compares neighbourhoods. All three change completely if one feature is expressed in euros and another in thousands of euros. Unlike some supervised methods (e.g., decision tree), these methods cannot compensate for a bad scaling.

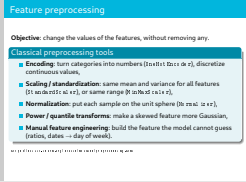
Feature preprocessing

Objective: change the values of the features, without removing any.

Classical preprocessing tools

- **Encoding**: turn categories into numbers (`OneHotEncoder`), discretize continuous values,
- **Scaling / standardization**: same mean and variance for all features (`StandardScaler`), or same range (`MinMaxScaler`),
- **Normalization**: put each *sample* on the unit sphere (`Normalizer`),
- **Power / quantile transforms**: make a skewed feature more Gaussian,
- **Manual feature engineering**: build the feature the model cannot guess (ratios, dates → day of week).

`https://scikit-learn.org/stable/modules/preprocessing.html`



Examples of feature preprocessing methods.

Do not mix up the scaling and normalization: scaling acts *column-wise* (per feature, over the dataset), normalization acts *row-wise* (per sample, dividing by its norm). Normalization is what you want when only the direction of the vector matters, e.g. word counts in a document, where long documents should not end up far from everything.

Feature selection

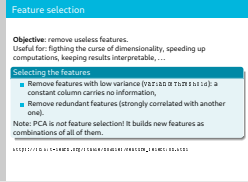
Objective: remove useless features.
Useful for: figthing the curse of dimensionality, speeding up computations, keeping results interpretable, …

Selecting the features

- Remove features with low variance (`VarianceThreshold`): a constant column carries no information,
- Remove redundant features (strongly correlated with another one).

Note: PCA is *not* feature selection! It builds new features as combinations of all of them.

`https://scikit-learn.org/stable/modules/feature_selection.html`



Selection differs from preprocessing in one respect: it *destroys* information, so it must be justified. The good reasons are on the slide: fewer dimensions means less overfitting for the next model, faster computations, and a result you can explain to a domain expert ("these 5 measurements are enough").

Without labels you can only remove what is constant or redundant. If you do have labels, other criteria exist (`SelectKBest`, or the feature importances of a model), but consist in supervision, even if weaker than full supervision (like using a supervised model end-to-end). Selection must be considered as part of the model.

Plan

1 Clustering

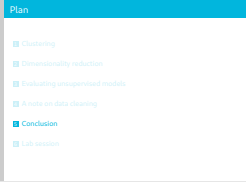
2 Dimensionality reduction

3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session



Conclusion: take-home messages

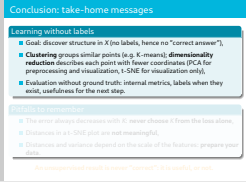
Learning without labels

- Goal: discover structure in X (no labels, hence no "correct answer"),
- **Clustering** groups similar points (e.g. K-means); **dimensionality reduction** describes each point with fewer coordinates (PCA for preprocessing and visualization, t-SNE for visualization only),
- Evaluation without ground truth: internal metrics, labels when they exist, usefulness for the next step.

Pitfalls to remember

- The error always decreases with K: **never choose K from the loss alone**,
- Distances in a t-SNE plot are **not meaningful**,
- Distances and variance depend on the scale of the features: **prepare your data**.

An unsupervised result is never "correct": it is useful, or not.



Let's wrap up.
Unsupervised learning is first of all a tool to *understand your data*: which points go together, how many dimensions really matter, what a 2D picture of the dataset looks like. In a real project, it usually comes before (or alongside) a supervised model: to explore the data, to clean it, or to provide better features. It is also very useful to evaluate and interpret neural networks embeddings (see future classes!)

The pitfalls are the mistakes most often seen in lab reports and projects: choosing K because the error is the smallest, commenting on the distance between two groups in a t-SNE plot, and running K-means or PCA on unscaled features. Avoid them, and justify each of your choices.

Conclusion: take-home messages

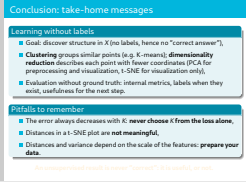
Learning without labels

- Goal: discover structure in X (no labels, hence no "correct answer"),
- **Clustering** groups similar points (e.g. K-means); **dimensionality reduction** describes each point with fewer coordinates (PCA for preprocessing and visualization, t-SNE for visualization only),
- Evaluation without ground truth: internal metrics, labels when they exist, usefulness for the next step.

Pitfalls to remember

- The error always decreases with K: **never choose K from the loss alone**,
- Distances in a t-SNE plot are **not meaningful**,
- Distances and variance depend on the scale of the features: **prepare your data**.

An unsupervised result is never "correct": it is useful, or not.



Let's wrap up.
Unsupervised learning is first of all a tool to *understand your data*: which points go together, how many dimensions really matter, what a 2D picture of the dataset looks like. In a real project, it usually comes before (or alongside) a supervised model: to explore the data, to clean it, or to provide better features. It is also very useful to evaluate and interpret neural networks embeddings (see future classes!)

The pitfalls are the mistakes most often seen in lab reports and projects: choosing K because the error is the smallest, commenting on the distance between two groups in a t-SNE plot, and running K-means or PCA on unscaled features. Avoid them, and justify each of your choices.

Conclusion: take-home messages

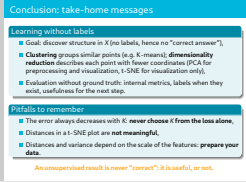
Learning without labels

- Goal: discover structure in X (no labels, hence no "correct answer"),
- **Clustering** groups similar points (e.g. K-means); **dimensionality reduction** describes each point with fewer coordinates (PCA for preprocessing and visualization, t-SNE for visualization only),
- Evaluation without ground truth: internal metrics, labels when they exist, usefulness for the next step.

Pitfalls to remember

- The error always decreases with K: **never choose K from the loss alone**,
- Distances in a t-SNE plot are **not meaningful**,
- Distances and variance depend on the scale of the features: **prepare your data**.

An unsupervised result is never "correct": it is useful, or not.



Let's wrap up.
Unsupervised learning is first of all a tool to *understand your data*: which points go together, how many dimensions really matter, what a 2D picture of the dataset looks like. In a real project, it usually comes before (or alongside) a supervised model: to explore the data, to clean it, or to provide better features. It is also very useful to evaluate and interpret neural networks embeddings (see future classes!)

The pitfalls are the mistakes most often seen in lab reports and projects: choosing K because the error is the smallest, commenting on the distance between two groups in a t-SNE plot, and running K-means or PCA on unscaled features. Avoid them, and justify each of your choices.

Plan

1 Clustering

2 Dimensionality reduction

3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session

Plan
Clustering
Dimensionality reduction
Evaluating unsupervised models
A note on data cleaning
Conclusion
Lab session

Lab Session 3: Unsupervised Learning

Part 1 — Toy datasets, with `sklearn`

- **Generate toy datasets** (blobs, moons, ...),
- Preprocess/select features, then cluster with `KMeans`,
- Reduce the dimension and visualise with `PCA` and `TSNE`,
- Find **where K-means breaks**, and try a different clustering method fitting the shape of the data.

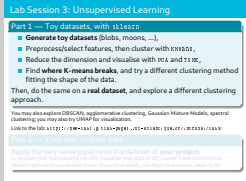
Then, do the same on a **real dataset**, and explore a different clustering approach.

You may also explore DBSCAN, agglomerative clustering, Gaussian Mixture Models, spectral clustering; you may also try UMAP for visualisation.

Link to the lab: `https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab3/`

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
⇒ prepare your features and say why, visualise your data in 2D, cluster it and confront the clusters with what you already know. If you have labels, use them to *evaluate*, never to fit.



Part 1 is the standard application of the principles of the course. There is no "best" unsupervised algorithm, only algorithms whose assumptions match (or not) the shape of your data. For instance, two intertwined moons are trivial for a density-based method and impossible for K-means, whatever you tune. Build the datasets that break the model on purpose, it is the fastest way to understand it!

Part 2 will help you better understand your data and project. You should keep your notebook and your conclusions, it may be useful for your final report!

The most useful outcome for most projects is the 2D visualization: it is the fastest way to notice that your data is duplicated, unbalanced, dominated by a single feature, or already separated. Colour your projection with everything you have (a category, a date, a source) even when it is not your target.

This lab is exploratory: the quality of the exploration and of the justification matters more than the beauty of the final plot. A negative result that you correctly diagnose is worth more than a lucky one.

Lab Session 3: Unsupervised Learning

Part 1 — Toy datasets, with `sklearn`

- **Generate toy datasets** (blobs, moons, ...),
- Preprocess/select features, then cluster with `KMeans`,
- Reduce the dimension and visualise with `PCA` and `TSNE`,
- Find **where K-means breaks**, and try a different clustering method fitting the shape of the data.

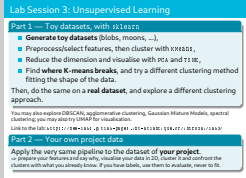
Then, do the same on a **real dataset**, and explore a different clustering approach.

You may also explore DBSCAN, agglomerative clustering, Gaussian Mixture Models, spectral clustering; you may also try UMAP for visualisation.

Link to the lab: `https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab3/`

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
⇒ prepare your features and say why, visualise your data in 2D, cluster it and confront the clusters with what you already know. If you have labels, use them to *evaluate*, never to fit.



Part 1 is the standard application of the principles of the course. There is no "best" unsupervised algorithm, only algorithms whose assumptions match (or not) the shape of your data. For instance, two intertwined moons are trivial for a density-based method and impossible for K-means, whatever you tune. Build the datasets that break the model on purpose, it is the fastest way to understand it!

Part 2 will help you better understand your data and project. You should keep your notebook and your conclusions, it may be useful for your final report!

The most useful outcome for most projects is the 2D visualization: it is the fastest way to notice that your data is duplicated, unbalanced, dominated by a single feature, or already separated. Colour your projection with everything you have (a category, a date, a source) even when it is not your target.

This lab is exploratory: the quality of the exploration and of the justification matters more than the beauty of the final plot. A negative result that you correctly diagnose is worth more than a lucky one.